

[Dashboard](#) / [Kurse](#) / [TNF](#) / [Institut für Machine Learning](#) / [2023S365114/246/247/248](#) / [Concluding Remarks](#) / [Exam](#)

Begonnen am Monday, 26. June 2023, 14:00

Status Beendet

Beendet am Monday, 26. June 2023, 14:13

Verbrauchte Zeit 13 Minuten 33 Sekunden

Frage 1

Richtig

Erreichbare Punkte: 1,00

What is the best possible description for the output of the following snippet of code?

```
@torch.enable_grad()
def compute_loss(network: nn.Module, loss_func: nn.Module, x: torch.Tensor, y: torch.Tensor):
    logits = network(x)
    err = loss_func(logits, y)
    err.backward()
    return err.item()
```

Wählen Sie eine Antwort:

- ☐ a. A `torch.Tensor` object with gradients
- ☐ b. A Python `list` object without gradients
- ☐ c. A Python `list` object with gradients
- ☐ d. A `torch.Tensor` object without gradients
- ☒ e. A Python `float` object without gradients ✓

Frage 2

Falsch

Erreichbare Punkte: 1,00

What is the most accurate description for the following snippet of code?

```
net = nn.Sequential()
net.add_module("features", FeatureNetwork(...))
net.add_module("classifier", nn.Linear(...))

state_dict = torch.load(...)
net.load_state_dict(state_dict)

net.features.requires_grad_(False)
net.classifier.requires_grad_(False)
```

Wählen Sie eine Antwort:

- ☐ a. Preparation of a network with pre-trained features for transfer learning (without feature fine-tuning).
- ☒ b. Preparation of a network with pre-trained features for transfer learning with feature fine-tuning. ✗
- ☐ c. Preparation of a fully pre-trained network for transfer learning (without feature fine-tuning).
- ☐ d. Preparation of a fully pre-trained network for transfer learning with feature fine-tuning.
- ☐ e. Preparation of a network that can not be used for transfer learning

Frage 3

Teilweise richtig

Erreichbare Punkte: 2,00

Which *two* of the following concepts do **not** belong in the list of answer options? Concepts are said to belong together if they fit within one of the assignments you made this semester.

Wählen Sie eine oder mehrere Antworten:

- ☐ a. reconstruction
- ☐ b. `model.load\_state\_dict(state\_dict)`
- ☒ c. residual networks ✓
- ☒ d. KL-divergence regularisation ✗
- ☐ e. transposed convolutions

Frage 4

Richtig

Erreichbare Punkte: 1,00

Which of the following lines of code implements the *projection* of projected gradient ascent, which is used to compute adversarial examples?

Wählen Sie eine Antwort:

- ☐ a. `delta = torch.zeros_like(x, requires_grad=True)`
- ☐ b. `delta += 0.1 * delta.grad.sign()`
- ☐ c. `logits = network(x + delta)`
- ☒ d. `delta.clamp_(-0.1, 0.1)` ✓
- ☐ e. `err.backward()`

Frage 5

Richtig

Erreichbare Punkte: 1,00

The code snippet below aims to produce the **objective to be minimised** for the generator of a GAN. You can assume the label for real (fake) labels to be one (zero). What is wrong with this code?

```
def generator_error(gan: nn.ModuleDict, x: torch.Tensor):
    mse = nn.MSELoss()
    z = torch.randn_like(x)
    with torch.enable_grad():
        fakes = gan.generator(z)
        logits = gan.discriminator(fakes)
        real_labels = torch.ones_like(logits)
        err = mse(fakes, x)
    return err
```

Wählen Sie eine Antwort:

- ☐ a. The generator and discriminator have been swapped.
- ☐ b. The generator will be unable to receive weight updates.
- ☐ c. The generator will always produce the same outputs during training.
- ☐ d. The generator weight updates will have the wrong sign.
- ☒ e. The generator error is computed with the wrong kind of loss function. ✓

Frage 6

Richtig

Erreichbare Punkte: 1,00

How would you implement the forward pass of an additive coupling layer?

☐ a.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    scale = self.conditioner(x2)  
    y1, y2 = x1 * scale, x2  
    return torch.cat([y1, y2], dim=1)
```

☐ b.

```
def coupling(y):  
    y1, y2 = torch.chunk(y, 2, dim=1)  
    scale = self.conditioner(y2)  
    x1, x2 = y1 / scale, y2  
    return torch.cat([x1, x2], dim=1)
```

☒ c.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    shift = self.conditioner(x2)  
    y1, y2 = x1 + shift, x2  
    return torch.cat([y2, y1], dim=1)
```

☐ d.

```
def coupling(x):  
    x1, x2 = torch.chunk(x, 2, dim=1)  
    shift = self.conditioner(x2)  
    y1, y2 = x1 + shift, x2  
    return torch.cat([y1, y2], dim=1)
```

☐ e.

```
def coupling(y):  
    y2, y1 = torch.chunk(y, 2, dim=1)  
    shift = self.conditioner(y2)  
    x1, x2 = y1 - shift, y2  
    return torch.cat([x1, x2], dim=1)
```

Frage 7

Falsch

Erreichbare Punkte: 3,00

Match the code snippets to the corresponding ML research papers.

Hint: Try to find algorithms, figures or equations in the papers, there should be no need to read them.

Click on the selected ML research paper to view it:

- ▶ **Going Deeper with Convolutions**
- ▶ **Rethinking the Inception Architecture for Computer Vision**
- ▶ **Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning**

```
import torch
from torch import nn

class
InceptionBlock(nn.Module):

    def __init__(self,
in_channels, o1x1_channels,
o3x3_channels, o5x5_channels,

pool_channels,
h3x3_channels=None
```

Going Deeper with Convolutions



```
import torch
from torch import nn

class
InceptionBlock(nn.Module):

    def __init__(self,
in_channels, o1x1_channels,
o3x3_channels, o5x5_channels,

pool_channels,
h3x3_channels=None
```

Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning



```
import torch
from torch import nn

class
InceptionBlock(nn.Module):

    def __init__(self,
in_channels, o1x1_channels,
o3x3_channels, o5x5_channels,

pool_channels,
h3x3_channels=None
```

Rethinking the Inception Architecture for Computer Vision

[◀ Final Question](#)

Direkt zu: